

RLVP: Penalize the Path, Reward the Outcome

Bojie Li
Pine AI

Noah Shi
University of Washington

Abstract

Reinforcement learning from verifiable rewards (RLVR) trains agents on the one signal an environment checks cheaply: whether the task succeeded. The instinct to go denser is to reward the *process*, but this reaches for the wrong signal. Agentic environments are asymmetric verifiers: they cheaply confirm an agent did something *bad* (a destructive command, a call before its precondition) but not that it made *progress*. The dense reward practitioners want is on the hard-to-verify side; the easy side is a *penalty*.

We make this precise with a within-group-variance account. A group-relative advantage is, by definition, a within-group variance, and it vanishes on all-failing groups early in training and all-succeeding groups late. A dense signal helps only where it supplies the variance the outcome lacks, and only where the policy reaches the states that carry it. A verifiable penalty meets this by construction—bad behavior is easy to sample and check—while a dense potential does not, and its speed-ups do not survive re-seeding.

This yields a simple recipe—**penalize the path, reward the outcome**—where the outcome reward drives the task while a per-action verifiable penalty teaches the outcome-neutral constraints that decide whether an agent is deployable. Across tasks and scales it attains the task with near-zero violations where outcome-only training violates every episode, and cuts harmful actions several-fold at *equal* success on a real benchmark. We give four rules for the penalty—including that a lone penalty stalls in an *inaction trap*—and show the mirror image: dense potentials are fragile where the account predicts.

Code: <https://github.com/19PINE-AI/rlvp>

Website: <https://01.me/research/rlvp>

1 Introduction

The most capable agents we have are coding agents, and on the benchmarks that made them famous they are very good: given a repository and a failing test suite, a frontier model reads the code, writes a patch, and turns the suite green. Yet the qualities that decide whether such a patch belongs in a production codebase—is it safe, does it respect the architecture, will it break something downstream?—are exactly the ones these agents still lack. In a controlled industry study of a real feature, run across many model–framework combinations, functional correctness exceeded 80% for *every* combination, yet the same implementations scored far lower—and almost at random—on reliability, maintainability, performance, and compatibility: they mishandled exceptions, broke previously working functionality, and ignored the codebase’s own conventions [Hong, 2026]. Passing the tests and writing code one can live with are different achievements, and the gap is not closing on its own.

The gap is not mysterious: good engineering is a discipline on the *path*, not a property of the *outcome*. A capable agent reads the few files that matter instead of the whole tree, writes

and runs tests, opens a pull request against a branch rather than pushing to main, and re-checks a deployment before declaring victory. None of this is implied by a green test suite—and a model optimizing only for that suite can reach it by deleting the failing test, hard-coding the expected output, or “fixing” a database by dropping it and recreating an empty one. The outcome is achieved; the result is unsafe and unusable. Getting the right answer and getting it the right way are different objectives, and only the second makes an agent deployable.

This second objective is what the reward driving agentic RL cannot supply. Reinforcement learning from verifiable rewards (RLVR) trains on the one thing an environment checks cheaply—the final outcome [DeepSeek-AI, 2025, Lambert et al., 2024]—to which engineering discipline is invisible, and against which a harmful shortcut can even score higher. The instinct to go denser is to reward the *process*: score partial progress at each step [Lightman et al., 2024, Wang et al., 2024, Setlur et al., 2025, Yu et al., 2024, Feng et al., 2025]. But progress, like safety or maintainability, is the hard-to-verify direction. Agentic environments are asymmetric verifiers: they tell you instantly and for free that an agent did something *bad*—ran a destructive command, called a tool before its precondition, broke a protocol—but not that it is on the right track, because judging that is the whole problem the agent is trying to solve. To build a progress reward one must train a learned critic, which the policy then learns to fool [Cheng et al., 2025, Kazemnejad et al., 2024, Gao et al., 2023], or hand-craft a proxy; either way one manufactures signal on the hard-to-verify side of the asymmetry.

We take the asymmetry seriously (Figure 1). The dense signal an environment can actually supply is a penalty on the path, not a reward on progress—and the folklore against penalties is a wiring error. A penalty *alone* fails in a specific way: its cheapest maximizer is to stop acting—never move, never violate—so the policy collapses into a penalty-free but useless optimum we call the *inaction trap*. But a penalty is not meant to drive the task; the outcome reward does that. Wired as a second channel *alongside* the outcome—reward the outcome, penalize the path—the penalty teaches exactly the outcome-neutral constraints that engineering discipline is made of: do not destroy, respect the precondition, follow the protocol. These are process rewards in the sense the field wants, but delivered from the verifiable side of the ledger; they give an agent good engineering principles that a green test suite never could.

That this division of labor is a consequence rather than a heuristic follows from a single quantity. A group-relative advantage is, definitionally, a within-group variance; it is zero on the all-failing groups that dominate early training and on the all-succeeding groups that dominate late, so the outcome reward is blind at both ends. A dense process signal helps only insofar as it supplies within-group variance the outcome lacks, and only where the policy reaches the intermediate states that carry it. A verifiable penalty satisfies this by construction—bad behavior is easy to sample and to check—while a dense progress potential does not, which is why the potential’s much-reported speed-ups are fragile and, in our hands, do not survive re-seeding.

Contributions.

- **A within-group-variance account** of when a dense process signal can help group-relative agentic RL: it must supply reachable within-group variance the outcome lacks (§2). The account is symmetric—outcome-only RL is blind on all-fail *and* all-success groups—and it separates penalties (reliably reachable) from potentials (usually not).
- **Penalize the path, reward the outcome**: a two-channel recipe in which a per-action verifi-

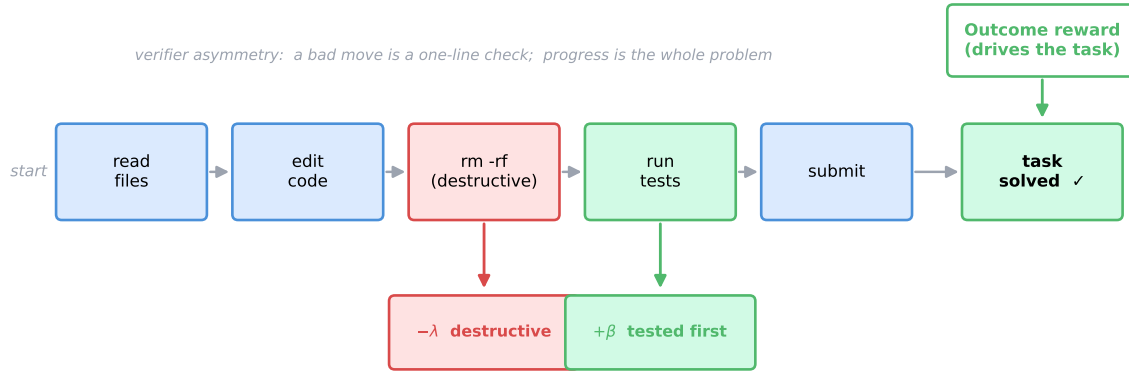


Figure 1. Reward the outcome, penalize the path. RLVR trains on the terminal outcome (right), which is blind to everything that happens along the trajectory. Agentic environments verify the *path* asymmetrically: they cheaply flag bad actions (a destructive command, a call without its precondition) but cannot certify progress. RLVP adds a second, per-action channel that penalizes those verifiable bad moves and discharges the compliant ones. The outcome channel drives the task; the penalty channel teaches the outcome-neutral constraints the outcome cannot see.

able penalty robustly teaches outcome-neutral constraints. Across diverse tasks and model scales it attains the task with near-zero violations where outcome-only training succeeds but violates every episode, and on a real agentic benchmark it cuts harmful actions several-fold at equal success (§3).

- **Four design rules for verifiable penalties**, each validated by ablation, including the inaction trap as the reason a penalty must accompany—never replace—the outcome reward (§4).
- **The mirror image**: dense progress potentials are fragile exactly where the variance account predicts—reachability gates them and their efficiency gains sign-flip across seeds (§5).

2 Background and When a Process Signal Helps

Group-relative RL. Modern agentic RL largely dispenses with a learned value function and estimates advantages by comparison *within a group* of rollouts of the same task [Shao et al., 2024, DeepSeek-AI, 2025]. For a prompt, one samples a group of G trajectories, scores each with a reward, and sets each trajectory’s advantage to its reward minus the group mean (often divided by the group standard deviation). There is no critic: the other members of the group *are* the baseline. This is what makes verifiable rewards so effective—a rule or a test suite scores a trajectory directly—and it is the setting for everything that follows.

The advantage is a within-group variance. Because the baseline is the group mean, a trajectory contributes gradient only to the extent its reward *differs* from its group-mates. If every rollout in a group receives the same reward, all advantages are zero and the group produces no update. Under a binary outcome reward this happens in two regimes (Figure 2): the *all-fail* groups that dominate early training on long-horizon tasks, and the *all-succeed* groups that dominate once a task is nearly solved. Group-relative RL is thus blind at both extremes of the success rate—a fact the field half-acknowledges by *discarding* such groups (dynamic sampling in DAPO drops prompts that are all-correct or all-wrong [Yu et al., 2025]) rather than

filling them with signal.

When a dense signal can help. A dense process signal is useful exactly when it restores variance the outcome has lost. Write the shaped reward of a trajectory as the outcome plus β times a process term, $R = O + \beta \Phi$. The group’s reward variance—the quantity that becomes gradient—then decomposes as

$$\text{Var}_G(R) = \text{Var}_G(O) + \beta^2 \text{Var}_G(\Phi) + 2\beta \text{Cov}_G(O, \Phi). \quad (1)$$

On an all-fail (or all-success) group the first term is zero, so *all* usable gradient must come from the process term’s own within-group variance $\text{Var}_G(\Phi)$. This gives a single, checkable condition: a process signal helps only where its within-group variance is nonzero—and that variance is realized only if the policy actually *reaches* differing intermediate states across the group. Two very different signals meet or miss this condition in opposite ways. A verifiable *penalty* on bad actions almost always carries variance: early in training some rollouts take the bad action and some do not, and both are cheap to sample and to check. A dense progress *potential* carries variance only when the policy reaches partial success—which, on the hard tasks where all-fail groups dominate, it usually does not. We confirm this directly in a real training run (Figure 2, right): the penalty channel carries within-group variance on precisely the all-fail groups where the outcome channel carries none, whereas a progress potential is near-zero there. The rest of the paper is this observation, made concrete: penalties are the reliable half of the process signal, potentials the fragile half.

3 Penalizing the Path: Robust Harm Reduction with Verifiable Penalties

Two channels. We keep the outcome reward and add a second, per-action channel (Figure 1). At each step, a deterministic rule engine—a pure predicate over the state before the step and the action taken—checks the action. A *violation* (a destructive command, a call issued before its required precondition) attaches a penalty $-\lambda$ to that action’s tokens; a *discharge* of a pending obligation (performing the precondition) attaches a credit $+\beta$. The two channels are normalized separately, so the sparse path signal is not diluted or cancelled by the outcome channel, then combined. “Penalize the path” is therefore literal: the gradient lowers the probability of the specific offending action, in the specific context where it occurred, while the outcome channel scales whole trajectories by success. We call such a signal a *verifiable penalty*: a deterministic predicate over the pre-action state and the action, checkable the moment it fires and independent of the eventual outcome. Crucially, the rules we penalize are *outcome-neutral*—following or breaking them does not change whether the task is solved—so this is signal the outcome reward can never provide. The full recipe pairs the penalty with its discharge, seeds a handful of scripted-compliant demonstrations so the compliant action is reachable, and anneals the shaping off once compliance saturates; §4 shows each of these earns its place.

The recipe attains the task with near-zero violations. We test on a suite of diverse system-administration and customer-service tasks whose outcome-neutral rules are exactly the engineering discipline of §1 in miniature: *do not submit work you have not tested, verify a precondition before you act, do not repeat a failed operation*. Drawn from a large pool and evaluated on held-out instances, outcome-only training learns to *solve* the tasks but violates these rules on essentially

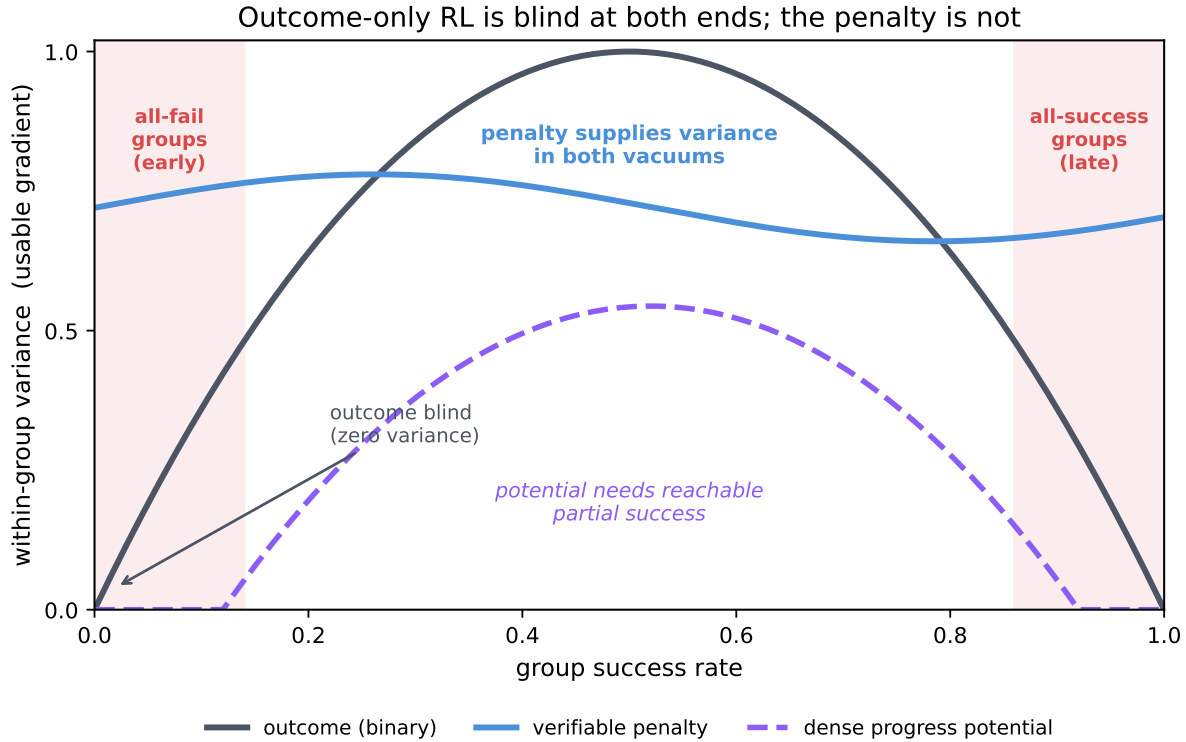


Figure 2. Outcome-only RL is blind at both ends of the success rate. A group-relative advantage is a within-group variance, which the binary outcome drives to zero on all-fail groups (early training) and all-succeed groups (late training). A dense process signal helps only by supplying within-group variance in these vacuums—and only where the policy reaches the intermediate states that carry it. A verifiable penalty (bad moves are easy to sample) meets this reliably; a dense progress potential (partial success is rare on hard tasks) does not.

every episode: the discipline is invisible to its reward, just as a green test suite is blind to whether the agent tested before it submitted. Adding the penalty channel—reward the outcome, penalize the path—drives violations to near zero while keeping task success high, across five seeds at 4B and holding from 1.7B to 8B (Figure 3). The agent does not achieve this by going quiet: it takes as many actions as before, simply cleaner ones. This is the positive result in its cleanest form: a verifiable penalty teaches a real, deployable behavior—the discipline itself—that the outcome reward cannot.

It transfers to a real agentic benchmark. The synthetic domains let us control the rules; to test the same mechanism in the wild we train on TerminalBench, where each turn is a real shell command in a task’s container and the environment flags destructive actions—the `rm -rf` and drop-the-database shortcuts of §1, now checked against a live filesystem. Task success there is at the floor for a small model, which makes it a clean test of the harm axis alone: at *equal* task success, the harm penalty cuts destructive actions several-fold relative to outcome-only training, while the policy issues *more* productive commands, not fewer (Table 1, five seeds). The reduction is large relative to its variance—the per-seed distributions do not overlap—and it is not passivity. A verifiable penalty bounds the path independently of whether the task is solved, which is exactly the property a deployable agent needs.

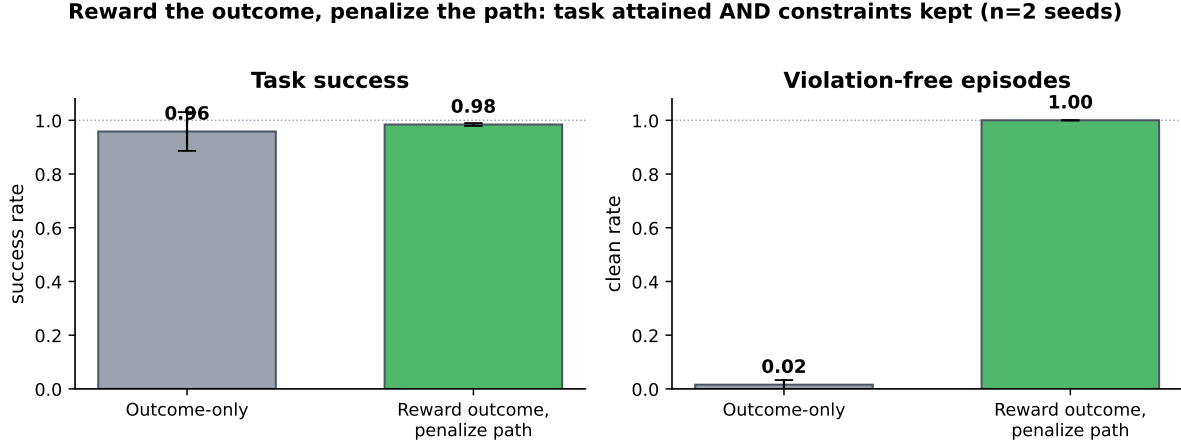


Figure 3. Reward-the-outcome-penalize-the-path attains the task *and* the constraints. Across diverse held-out tasks and model scales, outcome-only training solves the task but violates the outcome-neutral rules on nearly every episode (low “clean” rate). The two-channel recipe keeps task success high while driving violation-free episodes to near-100%, with small seed variance. Numbers are means over five seeds; error bars are one standard deviation.

Table 1. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 5 seeds, mean $\pm$ std). Task success is statistically equal and at the floor (4B rarely solves the benchmark; the small RLVP–outcome gap is within one standard deviation), yet the un-gameable harm penalty cuts harmful actions roughly *sixfold*. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	Outcome-only
Task success	0.097 ± 0.060	0.122 ± 0.076 (equal within 1σ , at floor)
Violations / episode	0.66 ± 0.63	3.71 ± 0.52
Productive actions / episode	~ 13	~ 4

4 Design Principles for Verifiable Penalties

A penalty is powerful but easy to misuse. The variance account and our ablations (Figure 5) yield four rules that, together, are the difference between the robust behavior above and a policy that stops working (Figure 4).

1. Penalize verifiable *actions*, not lack of progress. A good penalty targets a specific machine-checkable bad action—a destructive command, a call without its precondition. It must not target the *absence* of progress (“penalize an unproductive step”), because the cheapest way to make no unproductive steps is to make no steps. Penalize commission, never omission.

2. Keep the outcome reward as the task driver; never optimize a penalty alone. A penalty cannot be the primary learning signal. In the all-fail regime it is often the *only* signal with any slope, and its cheapest maximizer is to stop acting: the policy drifts into a penalty-free, useless optimum—the *inaction trap*. Our sweep makes this concrete: a pure penalty with no accompanying reward collapses to zero task success on every seed (Figure 5). The outcome reward is what forces the agent to keep acting; the penalty only shapes *how*. Reward the

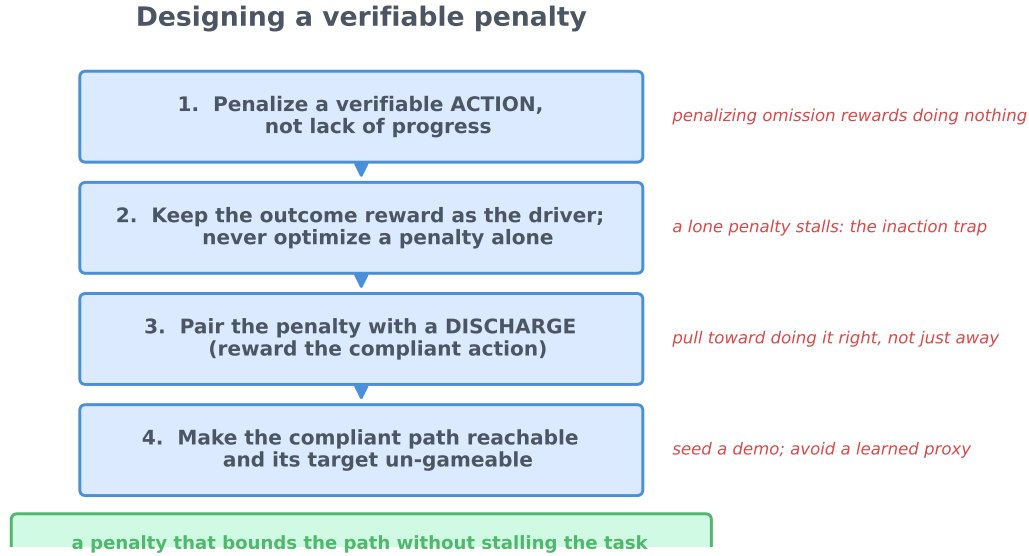


Figure 4. A design procedure for verifiable penalties. Before adding a penalty, a practitioner can check each rule in turn. Is there a verifiable *action* to penalize (not an absence of progress)? Is the outcome reward still the task driver? Is the penalty paired with a discharge for the compliant action? Is that compliant action reachable, and its target un-gameable? Only when all four hold does the penalty bound the path without stalling the task.

outcome, and the penalty becomes a passenger that steers rather than an engine that stalls.

3. Pair the penalty with a discharge. A penalty says only what *not* to do. Adding a discharge—a $+\beta$ credit for performing the compliant action—gives a positive pull *toward* doing it right, so the policy escapes the neighborhood of the bad action instead of merely avoiding it. Removing the discharge measurably slows and destabilizes the acquisition of compliant behavior (Figure 5).

4. Make the compliant behavior reachable and its target un-gameable. A discharge can only reward behavior the policy actually samples; if the compliant path is never explored, no amount of shaping fires. Seeding training with a few scripted-compliant demonstrations supplies that reachability, after which the shaping can be annealed off once compliance saturates. And the penalized (and discharged) target must be the real action, not a learnable or hand-wavy proxy: a *learned* judge of “compliance” simply relocates the hack into the judge, which the policy then games (Appendix ??).

Together these rules describe a narrow but reliable regime. The penalty is verifiable and outcome-neutral; it accompanies rather than replaces the outcome reward; it is paired with a discharge; and its target is reachable and un-gameable. The ablation in Figure 5 walks a policy from outcome-only, through a pure penalty (the inaction trap) and penalty-plus-discharge, to the full recipe, and the compliant-and-successful behavior appears only when all four rules hold.

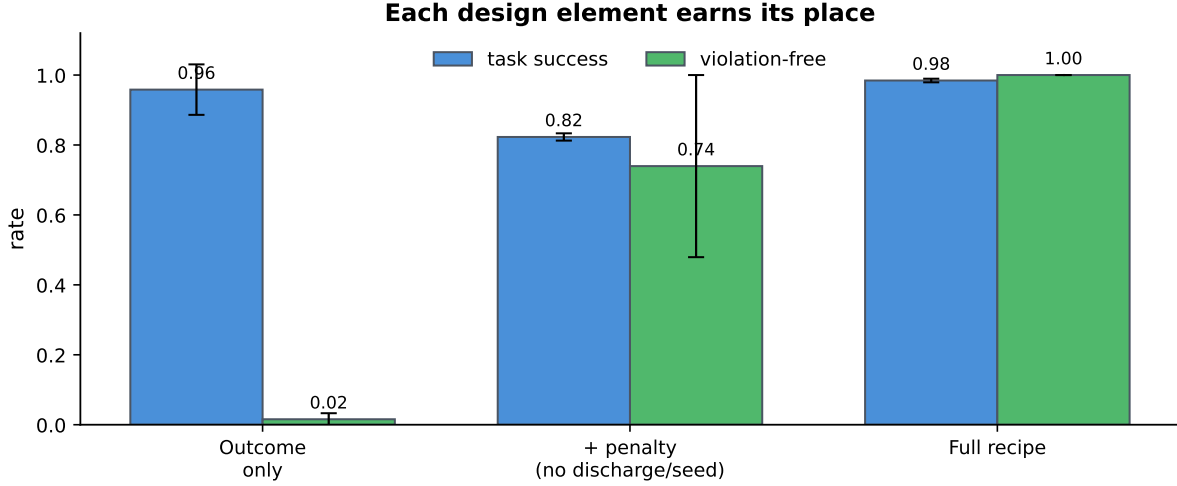


Figure 5. Each rule earns its place. Removing a design element degrades the outcome. Outcome-only leaves violations high; a *pure* penalty (no reward, no discharge) collapses to zero success—the inaction trap; penalty-plus-discharge without seeded reachability is seed-fragile; the full recipe attains high success *and* near-zero violations. Five seeds; bars show mean, whiskers one standard deviation.

5 The Fragility of Dense Potentials

Why not shape with a dense progress *potential* instead—rewarding the fraction of a proof completed, of tests passed, of stages cleared? We test this where a potential is most natural: theorem proving and software repair, tasks with a well-defined notion of partial progress, rather than the constraint-following domains above. The variance account predicts a potential is the fragile half of the process signal, and our experiments bear it out. The potential only supplies gradient where the policy reaches its intermediate values, and on the hard tasks where a dense signal is most wanted, it does not. On software repair the natural potential is the fraction of hidden tests an episode makes pass; across many rollouts of a capable model, *every* episode makes zero tests pass, so the potential’s within-group variance is identically zero and it centers out to nothing (Figure 6). This reachability, moreover, can be measured before training from a handful of base-policy rollouts, and the benefit of a dense potential tracks it.

Where the potential *is* reachable, it does supply gradient—but the resulting efficiency gains are not robust. A dense potential that appears to reach a hard theorem-proving benchmark several times faster than outcome-only training reverses under re-seeding: with the sampler’s randomness unfixed, the same configuration that “wins” on one run loses on the next, so even the *sign* of the speed-up is not reproducible (Figure 6). This is the agentic-process-reward instance of a broader fragility now documented for RLVR [Hochlehnert et al., 2025, Shao et al., 2025], and a sharper one. We therefore do not build the paper on a potential’s speed-up. The reliable use of the verifier is the penalty; the potential’s promise of acceleration is, at present, a lottery. Full experiments are in Appendix ??.

6 Discussion

Verifier asymmetry as a design principle. The practical takeaway is a reallocation. A verifier is a scarce, valuable thing, and the field has been spending it on the direction it is worst at—certifying progress—while avoiding the direction it is best at—catching bad moves. Spend

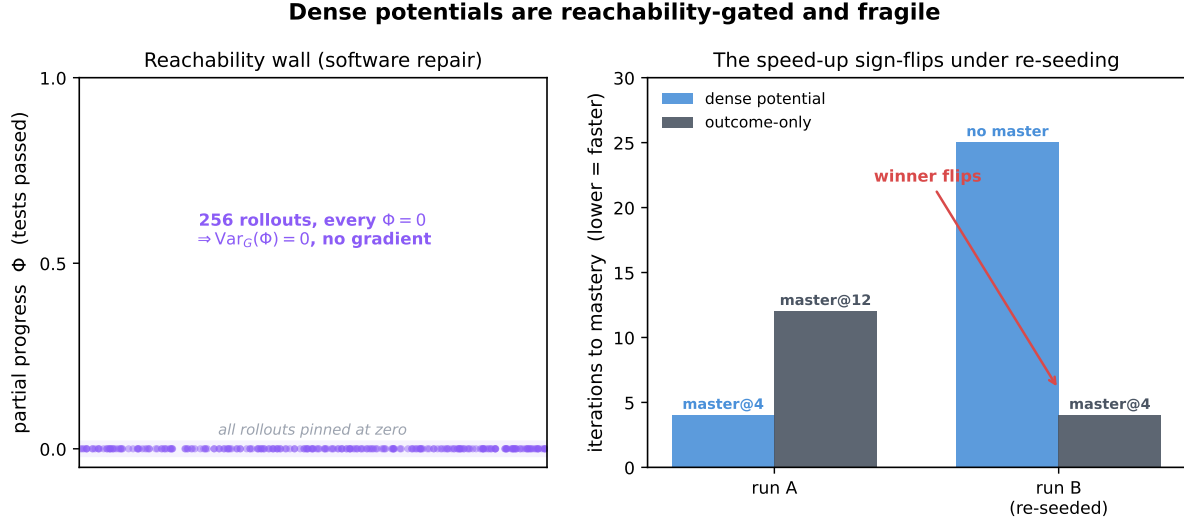


Figure 6. Dense potentials are reachability-gated and fragile. *Left:* on software repair the finer potential is unreachable—across many rollouts the policy makes zero partial progress, so its within-group variance is zero and it gives no gradient. *Right:* where a potential is reachable its apparent speed-up does not survive re-seeding—the same configuration reverses sign across runs. The reliable half of the process signal is the penalty, not the potential.

it on penalties. When there is an outcome-neutral constraint a deployed agent must respect, and a verifiable bad action to attach it to, a penalty paired with a discharge and wired alongside the outcome reward will teach it, robustly and cheaply. When the only dense signal available is a progress proxy, expect a lottery, and prefer to simply reward the outcome.

A systems note. The property that makes a verifiable penalty cheap—a machine-checkable environment—tends to make the verifier a CPU process (a rule engine, a test runner, a container). In our theorem-proving runs the accelerator generated a step in milliseconds and then waited seconds on the proof kernel, leaving GPU utilization in the single digits at full memory (Figure 7). This is workload-dependent, not a law, but for the verifiable domains where penalties apply, throughput is gated by environment parallelism, and the right systems design co-locates many verifier workers per accelerator rather than scaling accelerators.

Limitations. Our cleanest real-task harm result is measured where task success is at the floor; the mechanism predicts it holds at higher success (a penalty’s variance is reachable regardless of success), but demonstrating that on a capable model is the natural next step. Our penalties are hand-identified per domain from the environment’s rules; automating their discovery from an environment’s affordances is open. And the un-gameability sweep that anchors the inaction-trap result is high-variance at scale; we report its *survival pattern* rather than precise magnitudes.

7 Related Work

Group-relative RL and the zero-variance problem. Agentic RL has largely converged on GRPO [Shao et al., 2024] and the rule-based R1 line [DeepSeek-AI, 2025, Lambert et al., 2024],

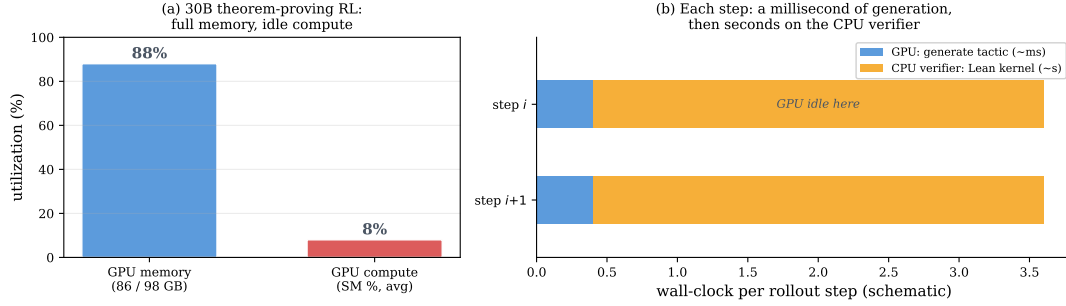


Figure 7. Verifiable agentic RL is often verifier-bound. In theorem-proving runs the GPU sits near-idle at full memory while the CPU proof kernel checks each step. The signal that makes a penalty cheap also moves the bottleneck off the accelerator.

which replace the value critic with a within-group baseline. Under binary verifiable rewards, an output’s advantage is then *definitionally* a within-group variance, and it collapses to zero whenever a group is uniformly solved or uniformly failed. The field has named this “advantage collapse” and responded in four ways: DAPO [Yu et al., 2025] *discards* zero-variance prompts via dynamic sampling, keeping only groups with $0 < \text{\#correct} < G$; Dr. GRPO [Liu et al., 2025] *removes* the difficulty-dependent standard-deviation normalizer; GSPO [Zheng et al., 2025] *stabilizes* the importance ratio at sequence level; and RL-ZVP [Le et al., 2025] *manufactures* a signal in zero-variance groups from output entropy. Notably, DAPO discards *all-correct* groups as well as *all-wrong* ones—an implicit acknowledgment that group-relative RL is blind at *both* extremes. We make the symmetry explicit and, rather than dropping the blind groups, ask which dense signal can *supply* the missing variance—finding that a verifiable penalty reliably can and a dense potential reliably cannot.

Process reward models and their gameability. Step-level supervision outperforms outcome supervision for verification [Lightman et al., 2024], with per-step labels obtainable automatically via Monte-Carlo rollouts [Wang et al., 2024]. Setlur et al. [2025] argue the useful process signal is a progress measure—a step-level advantage—closely paralleling our variance view, but instantiate it with a *learned* prover. A consistent body of evidence shows learned dense rewards are hacked under RL pressure: over-optimization follows clean Goodhart laws [Gao et al., 2023], capable agents exploit misspecification harder [Pan et al., 2022], summation-form PRM credit induces RL collapse [Cheng et al., 2025], and learned value credit is unreliable [Kazemnejad et al., 2024]. DeepSeek-R1 [DeepSeek-AI, 2025] abandoned PRMs because a model-based PRM “inevitably leads to reward hacking,” while Yuan et al. [2024a] show an outcome reward model implicitly contains a PRM. We take these as motivation for a *verifiable, non-learned* penalty that cannot be gamed by construction, and as evidence for our fourth design rule.

Potential-based shaping: safe versus useful. Ng et al. [1999] prove potential-based shaping leaves the optimal policy unchanged for *any* potential, and Wiewiora [2003] show such shaping equals value initialization—it affects learning dynamics, not the fixed point. This certifies which dense signals are *safe*, not which are *useful*. Our account is orthogonal to the safety guarantee: among safe shapings, the useful ones supply within-group variance the outcome lacks, and only where reachable—which is why a penalty on always-sampled bad actions

helps and a potential on rarely-reached progress does not.

Process rewards in agentic RL, and the fragility of RL gains. Step- and turn-level credit for agents has been explored via hierarchical critics [Zhou et al., 2024], learned step rewards [Yu et al., 2024, Wang et al., 2025a], and turn-level advantage estimators [Wei et al., 2025a]; the closest to a verifiable signal are GiGPO [Feng et al., 2025] and per-tool-call correctness [Qian et al., 2025]. Verifiable *outcome* rewards underpin agentic benchmarks and training [Yao et al., 2024, Barres et al., 2025, Pan et al., 2025, Wei et al., 2025b]. Wang and Ammanabrolu [2025] find dense turn-level rewards *accelerate* training but their effect on final performance depends on the policy-gradient estimator. This aligns with a broader reproducibility literature: pass@1 varies by up to $\sim 15\%$ across seeds [Hochlehnert et al., 2025], random rewards can “improve” some models through optimization bias [Shao et al., 2025], one example suffices for much of the gain [Wang et al., 2025b], and at large k base models match RLVR models [Yue et al., 2025]. Our potential result is the agentic-process-reward instance of this fragility, and sharper: even the *sign* of the speed-up flips under re-seeding. Learned-judge rewards [Yuan et al., 2024b, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023] and off-policy demonstration mixing [Yan et al., 2025] are complementary; our fourth design rule uses the latter for reachability and warns against the former for un-gameability.

8 Conclusion

RLVR taught language agents from the one bit an environment checks cheaply—the final outcome. The instinct to go denser has been to reward the process, but the environment cannot verify progress; it can only verify mistakes. The dense signal it actually affords is a penalty on the path, not a reward on progress. Wired correctly—alongside the outcome reward, paired with a discharge, aimed at a reachable and un-gameable target—a verifiable penalty teaches the outcome-neutral constraints that make an agent deployable, robustly and across scales, while a lone penalty stalls and a dense potential is a lottery. The contribution is not a new optimizer but a reallocation of a scarce resource: spend the verifier on penalties that bound the path, and let the outcome reward do what it already does—teach the task.

Acknowledgements

We thank Dingkun Hong, Dongjiang Li, and Wei Zhou of ByteDance for early discussions that motivated this research. This paper was produced using Pine Copilot’s voice-directed *whisper coding* workflow [Pine AI, 2025], in which the authors specify, discuss, and review the work by voice while a coding agent—Claude Code with Claude Opus 4.8—carries out the planning, coding, experiments, and paper writing. We thank BSQL Networking for hosting the NVIDIA RTX PRO 6000 GPU.

References

- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evalu-

- ating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- Jie Cheng et al. Stop summation: Min-form credit assignment is all process reward model needs for reasoning. *arXiv preprint arXiv:2504.15275*, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.
- Andreas Hochlehnert et al. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. *arXiv preprint arXiv:2504.07086*, 2025.
- Dingkun Hong. AI Coding: Practice and exploration, 2026. Keynote, ByteDance / Volcano Engine Force Conference, Beijing, June 2026. <https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg> (accessed 2026-06-25).
- Amirhossein Kazemnejad et al. VinePPO: Unlocking RL potential for LLM reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Thanh-Long V. Le, Myeongho Jeon, Kim Vu, Viet Lai, and Eunho Yang. No prompt left behind: Exploiting zero-variance prompts in LLM reinforcement learning via entropy-guided advantage shaping. *arXiv preprint arXiv:2509.21880*, 2025.
- Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIFF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let’s verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022.

- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. SWE-Gym: Training software engineering agents and verifiers. In *International Conference on Machine Learning (ICML)*, 2025.
- Pine AI. Pine AI: The most natural human-computer interface is your voice. Blog post, 2025. URL <https://www.19pine.ai/blog/pine-ai-the-most-natural-human-computer-interface-is-your-voice>. Accessed 2026-06-28.
- Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Amrith Setlur, Chirag Nagpal, et al. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Rulin Shao et al. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Hanlin Wang, Chak Tou Leong, Jiashuo Wang, Jian Wang, and Wenjie Li. SPA-RL: Reinforcing LLM agents via stepwise progress attribution. *arXiv preprint arXiv:2505.20732*, 2025a.
- Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Ruiyi Wang and Prithviraj Ammanabrolu. A practitioner’s guide to multi-turn agentic reinforcement learning. *arXiv preprint arXiv:2510.01132*, 2025.
- Yiping Wang et al. Reinforcement learning for reasoning in large language models with one training example. *arXiv preprint arXiv:2504.20571*, 2025b.
- Quan Wei, Siliang Zeng, Chenliang Li, William Brown, and Mingyi Hong. Reinforcing multi-turn reasoning in LLM agents via turn-level reward design. *arXiv preprint arXiv:2505.11821*, 2025a.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, others, and Sida I. Wang. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025b.
- Eric Wiewiora. Potential-based shaping and Q-value initialization are equivalent. *Journal of Artificial Intelligence Research (JAIR)*, 19:205–208, 2003.
- Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.
- Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. *arXiv preprint arXiv:2412.01981*, 2024a.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024b.
- Yang Yue et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- Chujie Zheng, Shixuan Liu, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.
- Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *International Conference on Machine Learning (ICML)*, 2024.